

TOPIC 01: OOPS IN JS & OBJECT PROTOTYPES

OOPs Ka Purpose:

Code ko modular, structured, clean aur reusable banane ke liye Object-Oriented Programming use hoti hai.

5 Pillars of OOPs in JS:

1. PROTOTYPES

2. NEW OPERATOR

3. CONSTRUCTORS

4. ES6 CLASSES

5. INHERITANCE (EXTENDS, SUPER)

Object Prototypes:

Prototypes wo mechanism hai jisse JavaScript objects ek doosre se properties aur methods ko **Inherit (prapt)** karte hain. Yeh ek single template object ki tarah kaam karta hai jisse sabhi objects bina apni alag copy banaye method use kar sakte hain.

- `arr.__proto__` → Array prototype object ka reference hota hai.
- `String.prototype` → Strings ke saare built-in methods (`trim`, `toUpperCase`) yahin hote hain.

TOPIC 02: FACTORY FUNCTIONS & INKA DRAWBACK

Factory Function Kya Hai?

Aisa simple function jo object banakar return karta hai:

```
function personMaker(name, age) { return { name: name, age: age, talk() { console.log(`Hi, I am ${this.name}`); } } }; let p1 = personMaker("Adam", 25); let p2 = personMaker("Eve", 25); console.log(p1.talk === p2.talk); // Output: FALSE (Memory Wastage!)
```



Factory Function Ka Nuksan: Har naye object ke liye memory me talk() method ki nayi duplicate copy banti hai! Iska solution hai **Constructor + Prototype**.

TOPIC 03: CONSTRUCTOR FUNCTIONS & 'NEW' OPERATOR

The 'new' Operator:

new operator kisi constructor function se naya object instance create karne ke liye use hota hai.

'new' Keyword Ke 4 Internal Steps:

1. CREATES EMPTY OBJECT {}



2. SETS 'THIS' = {}



4. RETURNS THE OBJECT



3. LINKS __PROTO__ TO PROTOTYPE

Constructor With Prototype Sharing:

```
// Constructor function (Capital letter se shuru hota hai) function Person(name, age)
{ this.name = name; this.age = age; } // Method ko prototype par add kiya (Memory
efficient) Person.prototype.talk = function() { console.log(`Hi, my name is
${this.name}`); }; let p1 = new Person("Adam", 25); let p2 = new Person("Eve", 25);
console.log(p1.talk === p2.talk); // Output: TRUE ✅ (Same memory reference!)
```



Key Takeaway: Prototype par method rakhne se laakhon objects bhi sirf ek hi method reference share karte hain!

TOPIC 04: ES6 CLASSES (MODERN SYNTAX)

What is Class?

ES6 Classes constructor functions aur prototypes ka ek clean, modern **Syntactic Sugar** hain.

```
class Person { constructor(name, age) { this.name = name; this.age = age; } talk() { console.log(`Hello, I am ${this.name}`); } } const p1 = new Person("Rahul", 21); p1.talk();
```

TOPIC 05: INHERITANCE (EXTENDS & SUPER)

Inheritance Concept:

Parent class ki properties aur methods ko Child class me bina code repeat kiye reuse karna.

- extends → Child class ko parent class se jodta hai.
- super(args) → Parent class ke constructor ko call karta hai (Parent properties initialize karne ke liye).

```
// Base/Parent Class class Person { constructor(name, age) { this.name = name; this.age = age; } } // Child Class (Inherits Person) class Student extends Person { constructor(name, age, marks) { super(name, age); // Parent constructor call this.marks = marks; } } const s1 = new Student("Aman", 20, 95); console.log(s1.name, s1.marks); // Aman 95
```



Rule: Child class ke constructor me this use karne se pehle super() call karna mandatory hota hai!

